

第27回 実践：Web ゲームプログラミング 6（得点・残機の表示）

実習時のフォルダ構成

Web_名前 の中に 27 フォルダを作る。第 26 回の `sample` フォルダと `exercise` フォルダを、27 の中にコピーして、続きから作る。

```
Web_名前
└── 27
    ├── sample          ← 第 26 回の sample をコピーする
    │   ├── index.html
    │   ├── style.css
    │   └── main.js
    └── exercise        ← 第 26 回の exercise をコピーする
        ├── index.html
        ├── style.css
        └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、`sample/main.js` に書き足す）
- `exercise`：自分のシューティングゲーム（演習 1～2、発展）

講義内容

`sample/main.js` に書き足す。書いたら `sample/index.html` をブラウザで開いて確認する。

1. 前回の復習

- `isHit(a, b)`：2 つの四角形が重なっていたら `true` を返す関数。
- `checkBulletHits`：`for` の入れ子で、すべての弾とすべての敵を比べる。当たったら両方を消して、得点を増やし、`break` で次の弾へ進む。

今回は、次の 2 つを行う。

- 敵が自機にぶつかったら、残機を 1 減らす。
- 得点と残機を、画面に表示する。

2. 今回の main.js の並び

```
// ===== 準備 =====  
// ===== ゲームの状態 ===== ← 残機の変数を追加する  
// ===== 自機・弾・敵 =====  
// ===== キー入力 =====  
// ===== 更新 ===== ← checkPlayerHit を追加し、update から呼び出す  
// ===== 描画 ===== ← drawUI を書き換える  
// ===== ゲームループ =====
```

3. 残機の変数

「ゲームの状態」の部分に、`lives` を追加する。

```
// ===== ゲームの状態 =====  
let score = 0;  
let lives = 3;  
let frameCount = 0; // ゲーム開始からのフレーム数
```

4. 敵と自機の当たり判定 (checkPlayerHit)

「更新」の部分の `checkBulletHits` の下に、`checkPlayerHit` を追加する。

```
// 敵と自機の当たり判定  
function checkPlayerHit() {  
  for (let i = enemies.length - 1; i >= 0; i--) {  
    if (isHit(player, enemies[i])) {  
      enemies.splice(i, 1);  
      lives -= 1;  
      break; // 1 フレームで減る残機は 1 つまで  
    }  
  }  
}
```

- 前回作った `isHit` を、そのまま使い回している。`player` も `enemies[i]` も、`x・y・w・h` を持つオブジェクトなので、同じ関数で判定できる。
- 自機は 1 つなので、`for` は入れ子にならない。すべての敵と、自機を比べる。
- ぶつかった敵を消して、残機を 1 減らす。
- 敵が 2 体同時にぶつかっても、残機が一度に 2 つ減らないように、`break` で `for` を終わらせる。

「更新」の部分の `update` を、次のようにする。

```
function update() {  
  frameCount += 1;  
  updatePlayer();  
  updateBullets();  
  updateEnemies();  
  checkBulletHits();  
  checkPlayerHit();  
}
```

5. 得点と残機を表示する (drawUI)

「描画」の部分の `drawUI` を、次のように書き換える。第 22 回から表示していた `FRAME`・`BULLETS`・`ENEMIES` は、動作確認のための表示だったので、ここで消す。

```
function drawUI() {
  ctx.fillStyle = "#ffffff";
  ctx.font = "20px sans-serif";
  ctx.textAlign = "left";
  ctx.fillText(`SCORE: ${score}`, 10, 30);
  ctx.fillText(`LIVES: ${lives}`, 10, 60);
}
```

- `fillText` の文字の部分に、テンプレートリテラルを使っている。`${score}` の部分に、そのときの得点が入る。
- `drawUI` は毎フレーム呼び出されるので、得点や残機が変わると、すぐに表示も変わる。
- `drawUI` は、`draw` の最後に呼び出している。最後に描くので、文字が敵や弾の下に隠れない。
- `ctx.textAlign = "left";` は、次回、別の場所で `"center"` に変えるため、ここで毎回 `"left"` に戻している。

6. 確認

- 敵を倒すと、`SCORE` が 100 ずつ増える。
- 敵にぶつかると、敵が消えて、`LIVES` が 1 減る。
- 今は、残機が 0 になってもゲームが続き、`LIVES` がマイナスになる。ゲームオーバーは、次回作る。

演習

`exercise/main.js` (自分のシューティングゲーム) に書く。

演習 1 得点と残機を表示する

- 講義と同じように、`lives`・`checkPlayerHit` を追加して、敵にぶつかると残機が減るようにする。
- `drawUI` を書き換えて、得点を左上、残機を右上に表示する (右上は、`textAlign` を `"right"` にし、`x` を `canvas.width - 10` にする)。
- 文字の色・大きさは、自分で決める。

演習 2 動作を確かめる

- わざと敵にぶつかって、残機が 1 つずつ減ることを確認する。
- 敵を倒して、得点が増えることを確認する。

発展 (任意)

- 残機の数だけ、小さな四角形を横に並べて描く (`for` 文で、`lives` 回繰り返す)。

- 敵を倒せずに、画面の下へ逃してしまったら、得点を 50 減らす。得点が 0 より小さくならないようにする。